

StatStayTimeMu [SimTalk]

Returns the medium duration of the time during which *Exporters/Workers* stayed at a single *importer* who is managed by the *Broker* designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.StatStayTimeMu → time
```

Return Value

The return value has the data type *time*.

Example

```
print MyBroker.StatStayTimeMu
```

See also

Statistics report, [Broker Statistics \[described\]](#)

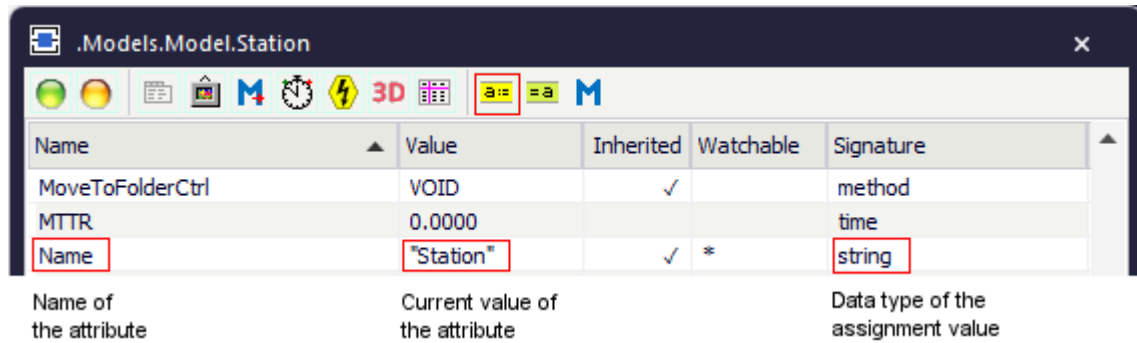
Attributes of the Broker

_Attributes of the Broker

The *Broker* provides:

- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).
- The [Attributes of the Material Flow Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyBroker.BrokerStatOn := true
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyBroker.ExpRequestCtrl  
posit := Station.Cont.XPos
```

BrokerStatOn [SimTalk]

Activates (`true`) or deactivates (`false`) statistics collection of the *Broker* designated by `<Path>`.

Type

Attribute

Syntax

```
<Path>.BrokerStatOn: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyBroker.BrokerStatOn := true
```

See also

[Broker Statistics \[activate\]](#)

ChooseNearestWorker [SimTalk]

Makes the *Broker* designated by <Path> prefer the *Worker*, who has to walk the **shortest path** to the *Workplace* (`true`). Specify `false` to make the *Broker* select any *Worker* who can do this job, no matter where he is located at the moment.

Remarks

When the *Broker* receives an **import request**, it selects *Workers*, who can fulfill the requested services according to the following criteria:

- Highest **Priority**
- Already located at a **Workplace** of the requesting station
- Matching **Scope**

When you specify `true` to *ChooseNearestWorker*, the *Broker* uses an **additional criterion** for selecting *Workers*: Which of the *Workers* has to walk a **shorter distance** to the *Workplace* than any of the other *Workers*.

Specify `false` to make the *Broker* select any *Worker* who can do this job, no matter where he is located at the moment.

Plant Simulation computes routes to the suitable *Workplaces* of the station for all eligible *Workers* while mediating them. For this reason, enabling *ChooseNearestWorker* can slow down the simulation.

Type

Attribute

Syntax

```
<Path>.ChooseNearestWorker : boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyBroker.ChooseNearestWorker := true
```

SimTalk

Scope [SimTalk] - Worker

[Priority \[SimTalk\] - Worker](#)

[See also](#)

[Choose the Nearest Worker \[check box\]](#)

[Priority \[Worker\]](#)

[Objects \[Scope\]](#)

[Workers to Create](#)

[How the Worker Decides Where to Work](#)

ExpRequestCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever an *Exporter/Worker* registers as available with its *Broker* so that it may be assigned a new *importer*.

Use the **Exporter Request Control** to assign a certain *importer* to the *Exporter/Worker*.

In the source code of the *method* you yourself have to make sure that the request is taken care of (for example with the method *engage*).

Type

Attribute

Syntax

```
<Path>.ExpRequestCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Example

```
MyBroker.ExpRequestCtrl := &myExporterRequestCtrl
```

SimTalk

[engage \[SimTalk\]](#)

See also

[Exporter Request Control \[Broker\]](#)

ImpRequestCtrl [SimTalk]

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever any *Broker* receives a request or when it would handle the request again, for example when an *Exporter/Worker* registers as available.

Use the **Importer Request Control** to model assignment strategies of your own.

In the source code of the *method* you yourself have to make sure that the request is taken care of (for example with the method *engage*).

Note

For *Broker* hierarchies *Plant Simulation* does not call the controls of the *sub-Brokers*, i.e., of several *Brokers* that are connected with *Connectors* in addition. *Plant Simulation* only calls the control of the *Broker* which you typed into the object.

Type

Attribute

Syntax

```
<Path>.ImpRequestCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Examples

```
MyBroker.ImpRequestCtrl := &myImporterRequestCtrl
```

```
-- The Broker tries to satisfy a request.
-- It can be an old open request
-- or a new request from an arriving part at the importer.
-- called by: control of the broker
param obj : object, -- Importer
type : integer -- Importer type (0=failure, 1=setup, 2=work)

var tab      :table[object, string, integer];var t:table
var service  : string;var j:integer
var part,expObj,allWorker: object
```

```

t.create
obj.imp.getAlternativeServices(t)
service := t[1,1][1,1]

part := @ --if part.id = 4 then debug
end
tab.create; allWorker := WorkerPartAssignment
if part /= void
  j := allWorker.getRowNo(part)
  if j = -1
    j := 1
    while j<= allWorker.yDim AND allWorker[0,j]/=void
      j := j + 1
    end
    if j<= allWorker.yDim
      allWorker[0,j] := part
      expObj := allWorker[1,j]
      tab.writeRow(1,1, expObj, service, 1 )
      broker.engage( obj, type, tab )
    end
  else
    expObj := allWorker[1,j]
    tab.writeRow(1,1, expObj, service, 1 )
    broker.engage( obj, type, tab )
  end
end
end

```

SimTalk

[engage \[SimTalk\]](#)

See also

[Importer Request Control \[Broker\]](#)

AGVPool

AGVPool

Use the object *AGVPool* for creating *automated guided vehicles* (AGVs). You can then model AGV systems in your installation in which the AGVs are not bound to a permanently installed route network.

Image

